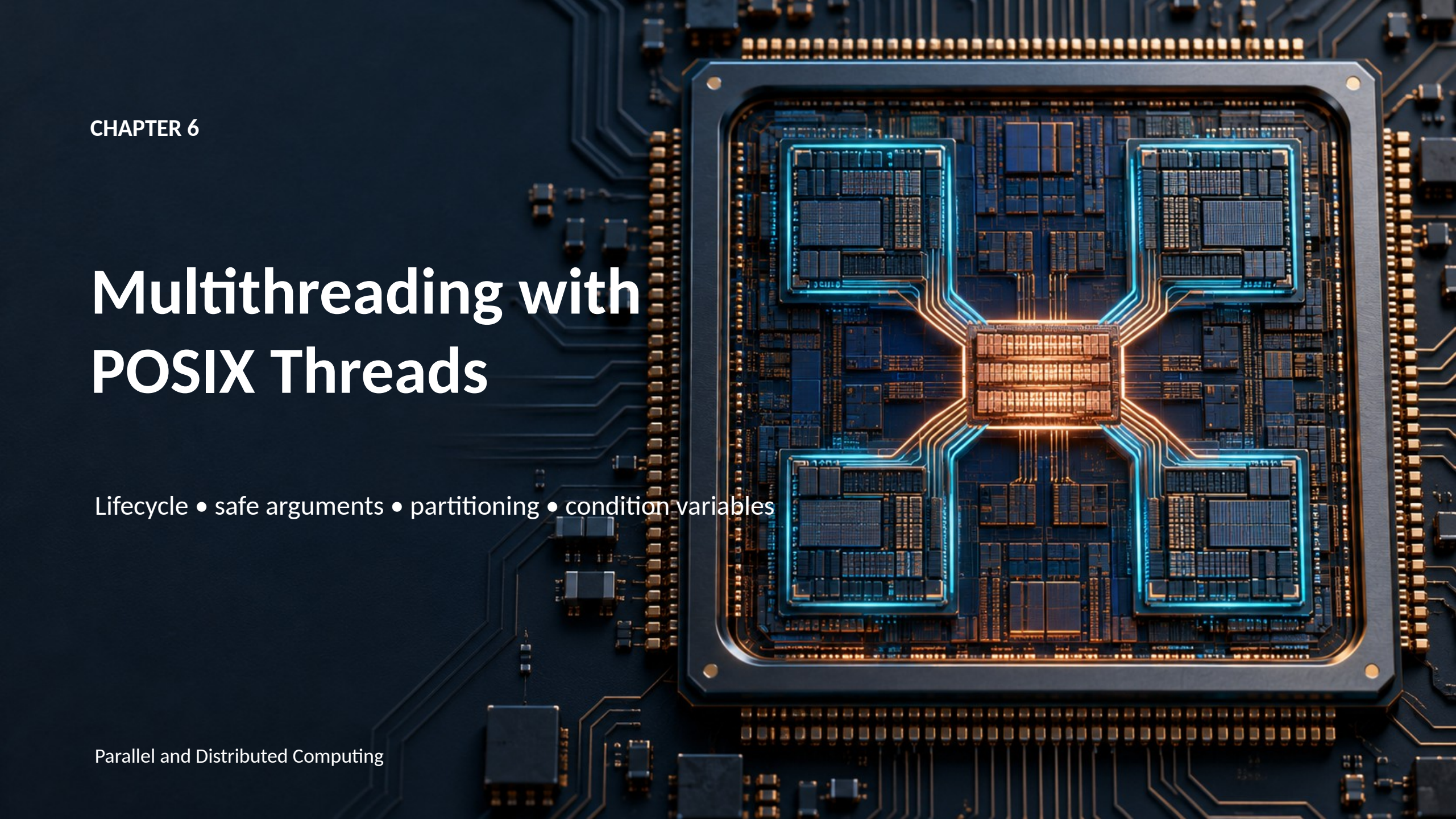
CHAPTER 6

Multithreading with POSIX Threads

Lifecycle • safe arguments • partitioning • condition variables



Parallel and Distributed Computing

How do we turn shared state into a correct program?

A thread can access shared memory immediately.

Correctness still needs three explicit agreements:

- Who owns each piece of work?
- When is a result complete?
- What state allows a waiting worker to proceed?

You will build and explain two complete programs

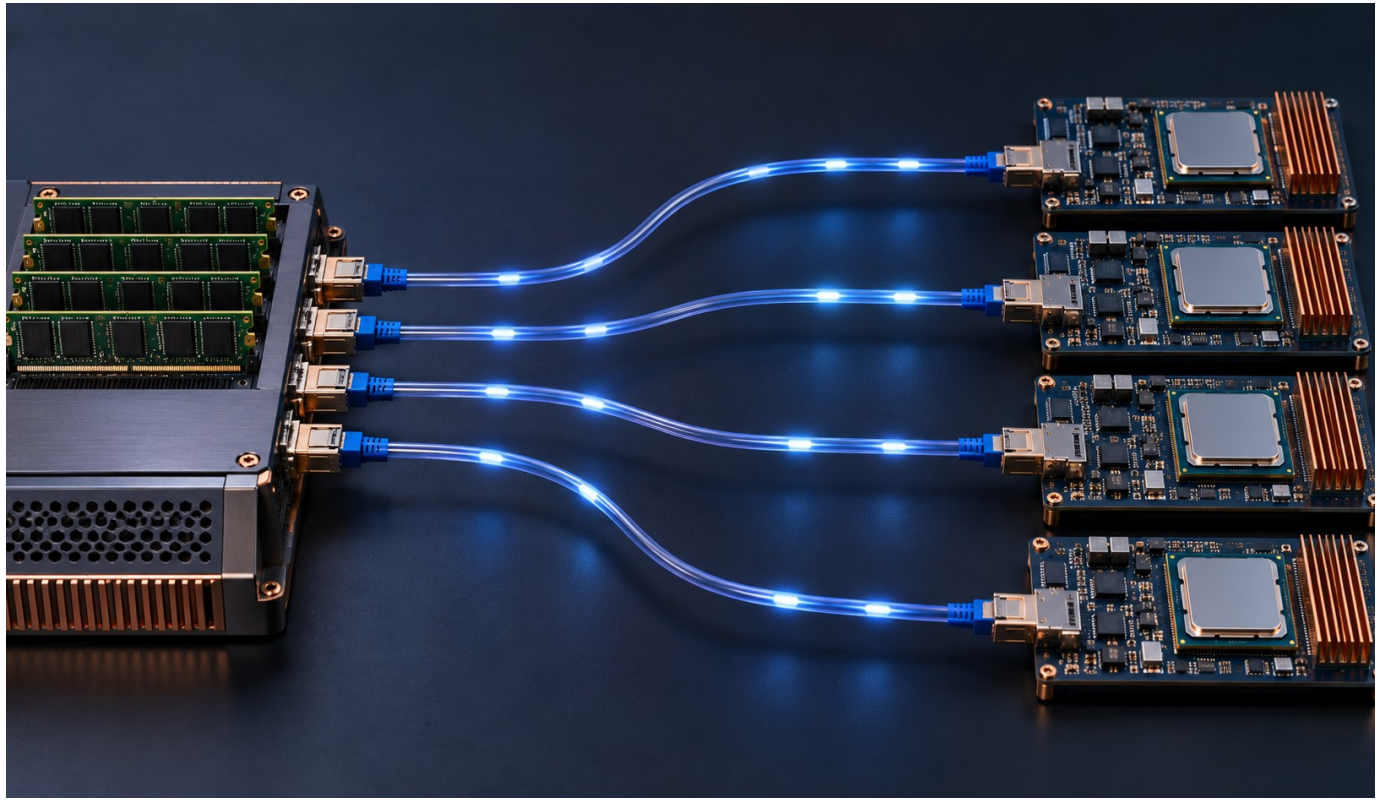
- A block-partitioned parallel sum with per-worker results
- A bounded producer–consumer queue with clean shutdown
- A correctness argument for every create, wait, and join

Chapter 5 gives the invariant; Pthreads gives the protocol

Question	Shared-memory rule	Pthreads action
Who may write?	Protect an invariant	Mutex or private record
When is data ready?	Establish ordering	Join or condition wait
How does work stop?	Define terminal state	Close, wake, drain, join

We focus on implementation mechanics; OpenMP and formal performance models follow later.

One process can run several threads



Shared

Address space and input array

Private

Stacks, loop indices, local sums

Conceptual illustration; actual threads share one process address space.

Create starts a worker; join confirms completion



The creator reads a worker result only after that worker has joined.

Five calls carry the chapter’s core protocol

Call	Use in this chapter
pthread_create	Start a worker with a stable argument
pthread_join	Wait and reclaim joinable-thread resources
pthread_mutex_lock / unlock	Guard queue state and predicates
pthread_cond_wait	Release mutex, sleep, reacquire, recheck
pthread_cond_signal / broadcast	Wake one or all waiters after state changes

The worker signature accepts one pointer

```
static void *sum_range(void *argument)
{
    Work *work = argument;
    /* Use work->begin, end, and data. */
    return NULL;
}
```

The pointer normally refers to a structure containing all worker inputs.

Join makes the result safe to read

```
pthread_create(&threads[t], NULL, sum_range, &work[t]);
/* Other workers may run here. */
pthread_join(threads[t], NULL);
total += work[t].partial;
```

Read before join → worker may still be writing.

Joinable and detached threads solve different jobs

Property	Joinable (default)	Detached
Completion result	Caller may retrieve it	No join result
Resource cleanup	Join once	Automatic on termination
Our examples	Yes: combine or clean up	No: lifetime still needs protocol

Detaching a thread does not make its input storage live longer.

Passing &t gives every worker the same address

```
for (int t = 0; t < P; ++t)
    pthread_create(&threads[t], NULL, worker, &t);
```

- The creator changes t while workers may read it.
- Observed IDs can repeat, skip, or race.

One Work record gives each worker stable inputs

```
typedef struct {
    const int *data;
    size_t begin, end;
    long long partial;
} Work;
Work work[WORKERS];
```

Fill work[t] before creating thread t; join before reading work[t].partial.

Ownership removes the need for a reduction lock

Object	Read by	Written by
data[0..N)	All workers	main before creation
work[t].begin/end	Worker t	main before creation
work[t].partial	main after join	Worker t only
total	main	main after joins

Distinct result records are separate locations, even when workers run together.

A good partition covers every index once

N = 1003 elements

P = 4 workers

The challenge:

Give workers nearly equal blocks, including the remainder, without gaps or overlaps.

Quotient and remainder define the bounds

$$N = qP + r$$

$q = \text{floor}(N/P)$ $\quad 0 \le r < P$

$\text{begin}(t) = tq + \min(t, r)$

$\text{end}(t) = (t+1)q + \min(t+1, r)$

The four ranges differ by at most one item

Worker t	begin	end	Length
0	0	251	251
1	251	502	251
2	502	753	251
3	753	1003	250

N = 1003, P = 4, q = 250, r = 3; ranges are half-open [begin, end).

The remainder changes only one worker’s load



Exact element counts from the worked partition; this is not a timing benchmark.

Three equalities prove full coverage

$$\text{begin}(0) = 0$$

$$\text{end}(t) = \text{begin}(t+1)$$

$$\text{end}(P-1) = N$$

Together: no gap, no overlap, no unassigned tail.

The worker sums only its assigned range

```
static void *sum_range(void *argument)
{
    Work *w = argument;
    long long local = 0;
    for (size_t i = w->begin; i < w->end; ++i)
        local += w->data[i];
    w->partial = local;
    return NULL;
}
```

The creator initializes each record first

```
for (size_t t = 0; t < WORKERS; ++t) {
    size_t q = N / WORKERS, r = N % WORKERS;
    work[t] = (Work){data,
        t*q + (t < r ? t : r),
        (t+1)*q + (t+1 < r ? t+1 : r), 0};
    pthread_create(&threads[t], NULL,
                   sum_range, &work[t]);
}
```

The complete file checks pthread_create's return value.

Join, then combine the private partial sums

```
long long total = 0;
for (size_t t = 0; t < WORKERS; ++t) {
    pthread_join(threads[t], NULL);
    total += work[t].partial;
}
printf("total=%lld\n", total);
```

Expected: 1 + 2 + ⋯ + 1003 = 503,506

A serial formula checks the final answer

Expected = N(N + 1) / 2

1003 × 1004 / 2 = 503,506

```
total=503506 expected=503506
```

The source program exits with failure if the two totals differ.

Private partials avoid a shared hot spot

Design	Shared updates	Coordination
One global sum	One per element (1003)	Lock or atomic each time
One partial per worker	Four partials combined after joins	No update inside worker loop

Counts are analytical for this example, not measured speedups.

A queue adds a different kind of coordination

What should a consumer do when the queue is empty?

- Busy-checking wastes CPU time.
- A condition variable lets it sleep.
- The queue predicate decides whether it may proceed.

The predicate, not the signal, is the fact

State under mutex → Wait or change state → Signal after change

A signal is not saved if nobody is waiting.

Condition wait releases and reacquires the mutex

```
pthread_mutex_lock(&queue.mutex);
while (queue.count == 0 && !queue.closed)
    pthread_cond_wait(&queue.not_empty,
                      &queue.mutex);
/* Mutex held again here. Recheck state. */
pthread_mutex_unlock(&queue.mutex);
```

The while loop handles spurious wakeups and competing consumers.

Signal and broadcast have distinct roles

Call	Who may wake?	Use here
signal(not_empty)	One waiting consumer	After an insertion
signal(not_full)	One waiting producer	After a removal
broadcast(not_empty)	All waiting consumers	After closing the queue

Waking a thread does not reserve the item or slot for it.

The bounded queue protects one shared state



head ● tail ● count ● closed ● values[]

One mutex protects these fields as a unit.

The queue state determines each action

State	Producer	Consumer
Empty, open	Insert	Wait for not_empty
Partly full	Insert	Remove
Full	Wait for not_full	Remove
Closed, still holds items	Finished	Drain items
Closed and empty	Finished	Stop

put waits only while the queue is full

```
pthread_mutex_lock(&queue.mutex);
while (queue.count == CAPACITY)
    pthread_cond_wait(&queue.not_full,
                      &queue.mutex);
queue.values[queue.tail] = value;
queue.tail = (queue.tail + 1) % CAPACITY;
++queue.count;
pthread_cond_signal(&queue.not_empty);
pthread_mutex_unlock(&queue.mutex);
```

take stops only after close and drain

```
pthread_mutex_lock(&queue.mutex);
while (queue.count == 0 && !queue.closed)
    pthread_cond_wait(&queue.not_empty, &queue.mutex);
if (queue.count == 0) {
    pthread_mutex_unlock(&queue.mutex);
    return 0;
}
*output = queue.values[queue.head];
queue.head = (queue.head + 1) % CAPACITY;
--queue.count;
pthread_cond_signal(&queue.not_full);
pthread_mutex_unlock(&queue.mutex);
return 1;
```

Close, wake, drain, then join

```
/* After producer inserts its final item: */
pthread_mutex_lock(&queue.mutex);
queue.closed = 1;
pthread_cond_broadcast(&queue.not_empty);
pthread_mutex_unlock(&queue.mutex);
/* main then joins producer and consumer. */
```

The consumer returns only when closed and count == 0.

A four-step trace explains clean shutdown

Step	count	closed	Consumer action
Start	0	0	Wait
Put 1, put 2	2	0	Remove 1
Producer closes	1	1	Remove 2
Queue drained	0	1	Return end-of-stream

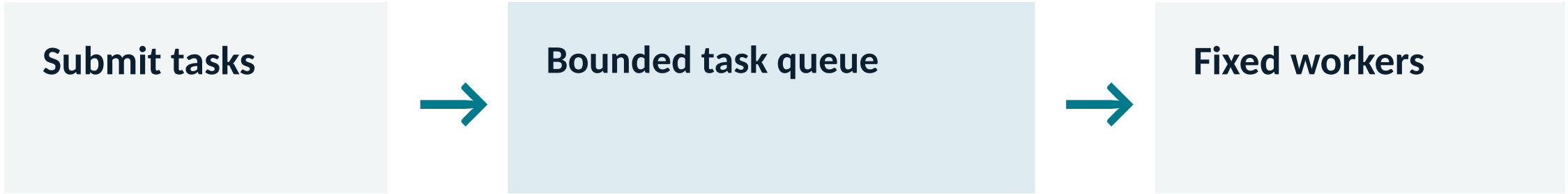
The consumer does not treat closed = 1 as permission to skip item 2.

More consumers need separate results

- Each consumer owns one sum field.
- Closing broadcasts to every waiting consumer.
- Main joins all consumers, then combines their sums.

A multi-producer version also needs a rule for who closes the queue.

Thread pools reuse workers across many tasks



Reuse avoids creating one thread for every short job.

The queue protocol is the core; a full pool also needs task results and shutdown rules.

Most failures are protocol mistakes

Symptom	Likely cause	Repair
Wrong worker IDs	Shared loop index address	Per-worker record
Wrong total	Read before join or bad range	Join and check bounds
Consumer hangs	Closed without wakeup	Broadcast on close
Empty after wake	Wait guarded by if	Recheck with while

Practice: partition 17 items among five workers

N = 17 P = 5

Find q and r, then write every half-open range.

Check: first begin = 0, adjacent endpoints match, last end = 17.

Practice: close a queue with two items left

count = 2 closed = 1

What do the next three calls to take return?

Explain why a waiting consumer must recheck both fields.

Correct Pthreads code makes lifetime and state explicit

- Create after initializing each argument record.
- Give workers disjoint work and private results.
- Join before reading results or freeing inputs.
- Wait in a loop on a mutex-protected predicate.
- Close, notify, drain, and join before cleanup.

Source code and API reference

Book source

Chapter 6: Multithreading with POSIX Threads

Runnable examples

examples/chapter6/parallel_sum.c

examples/chapter6/condition_queue.c

POSIX API: pubs.opengroup.org/onlinepubs/9799919799/functions/

Illustrations generated for this lecture; charts use calculated example values.